Interactive, Height-based Filtering in 2.5D Treemaps

Daniel Limberger
Hasso Plattner Institute
University of Potsdam
Potsdam, Germany
daniel.limberger@hpi.de

Matthias Trapp
Hasso Plattner Institute
University of Potsdam
Potsdam, Germany
matthias.trapp@hpi.de

Jürgen Döllner
Hasso Plattner Institute
University of Potsdam
Potsdam, Germany
juergen.doellner@hpi.de

ABSTRACT

The 2.5D treemap facilitates interactive exploration of multivariate data. It includes height as a visual variable for additional information display and enables exploration of correlations within mapped attributes. In this paper techniques for the visual display of a *height reference* for interactive modification from within the visualization are introduced. Results of a preliminary user study are presented and potential benefits for improved performance, i.e., precision and speed, of identification, comparison, filtering, and selection tasks are discussed.

CCS CONCEPTS

• **Human-centered computing** → **Treemaps**; • **Computing methodologies** → *Rendering*; • **Information systems** → Users and interactive retrieval;

KEYWORDS

Treemap, Height Reference, Height Cue, Filtering, Selection

ACM Reference Format:
Daniel Limberger, Matthias Trapp, and Jürgen Döllner. 2018. Interactive, Height-based Filtering in 2.5D Treemaps. In *VINCI '18: The 11th International Symposium on Visual Information Communication and Interaction, August 13–15, 2018, Växjö, Sweden.* ACM, New York, NY, USA, Article 4, 7 pages. https://doi.org/10.1145/3231622.3231638

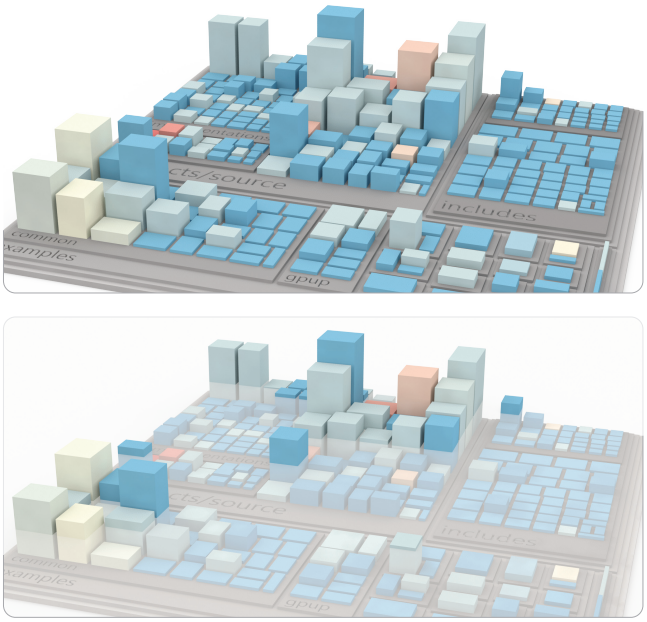


Figure 1: A softwaremap (top)—a 2.5D treemap depicting software system information—using a transparent plane for the visual display of height reference (bottom). A small open-source project is depicted, with the number of changes of a source code unit (color) correlated to the number of different developers per unit (height). The height reference allows for a precise identification of error-prone code units.

1 INTRODUCTION

A 2D treemap visualizes *tree-structured data elements* (nodes) and *related thematic information* (attributes) in a space-filling way [14], i.e., it maps attributes to graphical elements that are recursively stacked within their parents' elements. The visualization technique is applied in various research and industry applications to provide a comprehensive and consistent gestalt for communication and visual analytics [21] of non-spatial, massive, temporal and structural evolving information such as file systems [27] or software systems [2]. Furthermore, it is easy to implement and can be efficiently rendered in real-time [7].

The visualization and communication capabilities of 2D treemaps are limited compared to 2.5D treemaps that feature the third dimension [26]. They offer an additional dimension for extended information display—not only using height but also transparency or texture—and similar to 2D variants, can be efficiently rendered [29]. The added value of height, however, is limited by the treemap's overall appearance as well as the available interaction metaphors: we often find height to be either (1) reduced to a rough indication of data trends and correlations or (2) exposed to inaccurate user estimates with respect to the actual mapped values of meticulous resolution. Furthermore, a user's height estimate might be hindered due to perspective foreshortening of cuboids, as well as occlusion (Figure 2) or height distorting parent node representations [1].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
VINCI '18, August 13–15, 2018, Växjö, Sweden
© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.
ACM ISBN 978-1-4503-6501-7/18/08…$15.00
https://doi.org/10.1145/3231622.3231638

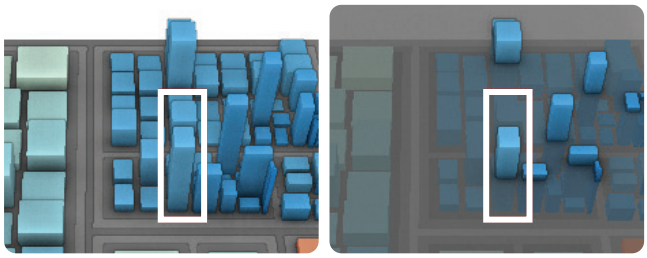


Figure 2: Two seemingly adjacent cuboids (outlined in white) of similar height (left). With a height reference applied a strong difference in height is exposed (right).

This paper introduces a *height-reference technique* that enables accurate comparison of heights across the treemap and precise selection and filtering of nodes-of-interest. For it, the following contributions are made:

- Multiple approaches for the visual display of height reference are introduced and discussed w.r.t. their applicability, usefulness, and implementation complexity.
- An interaction method for accurate, reference-based manipulation of the height reference within the treemap metaphor is presented.
- Results of a preliminary user study are presented, indicating that the use of the height reference increases accuracy for reading heights.

Height reference thereby denotes the depiction of an height threshold that can be interactively modified and enables height based queries. Within the context of this work, these details-on-demand related queries are tasks that enable accurate identification of nodes-of-interest based on their actual height. This comprises identification, filtering, selection, and comparison of nodes of specific, height-related characteristics:

Highest Nodes | Communication of order for identification of n top-most nodes.

Similar Nodes | Accentuation of nodes of similar or same height (peers).

Thresholded Nodes | Exclusion or inclusion of nodes below or above a threshold respectively.

We argue that the introduction of an interactive height reference to 2.5D treemaps (Figure 1) increases (1) accuracy of height readings, (2) allows for fast and precise identification of similar or correlating nodes, and (3) prevents miscommunication of height, e.g., caused by overlapping or occluding nodes of equally colored and shaded cuboids. Furthermore, we show results of our prototypical implementations, provide implementation details for web-based provisioning, and, finally, present results of a preliminary study we conducted.

2 RELATED WORK

Related work includes findings in the fields of visualization of tree-structured data, interaction, as well as rendering techniques and visual quality. For the remainder of this work the terminology of *inclusion-based, implicit hierarchy visualization* [4, 26] is applied.

2.1 Treemaps

Treemaps were introduced as two-dimensional, space-restricted visualization of tree-structured data in 1992 [27] and continue to increase in number of applications and enhancements ever since. Often used visual variables include size and color [7] as well as texture [11]. Other visual variables inherent or applicable to treemaps are location, size, shape, orientation, color hue, color value, color saturation, texture, arrangement/layout/order [25]. However, we advise against the simultaneous use of more than 3 to 4 visual variables in a map theme. For the specification of our map themes, we adhere to the information seeking mantra by mapping the most relevant attributes (for a given task) to visual variables that are subject to preattentive processing. The remaining visual variables are used for exploration and details-on-demand display. In order to increase recognition of the data's topology shading techniques [33] or, in our case study, margins or insets [20, 27] are used. For spatialization, we use a strip inverted layout [30] and are in the transition towards hybrid, map theme and, by implication, task aware layouting [8].

2.2 2.5D Treemaps

2.5D Treemaps provide additional information display for attribute mapping by extruding the graphical elements [1]. The graphical elements (*cuboids*) now consists of additional visual variables, e.g., height and transparency. The spatial reference, that is the location of every graphical element, remains unchanged and the third dimension is utilized for height display only, not for 3D positioning though. Thus, we refer to this extruded treemap as *2.5D treemaps* instead of 3D treemaps [31].

Albeit mapping leaf data to height is inherent, it is debatable whether or not graphical elements that represent inner nodes should have a height, e.g., depicting accumulated inner node data [2]. It was suggested [1] to increase the topological quality—a measure of the user performance and ability to read data structure, e.g., (1) associate leaf nodes to parents and (2) differentiate inner nodes by depth and adjacency—but aggravates height comparison of graphical elements of different parents and often increases the overall visual complexity. We argue to restrict height mapping exclusively to aggregated inner nodes [17] and increase the topological quality by an integral, well-tuned use of color schemes, contouring [15], labeling, and margins instead.

For a comprehensive overview of variations of treemaps and similar visualization techniques we refer to the survey by Schulz et al. [26]. Our height-reference technique is applicable to 2.5D visualizations such as 2.5D hierarchical edge bundles [10] and 2.5D icicle plots [16] as well as treemaps that limit the use of 3D to the exposition of additional detail in areas-of-interest [3, 18]. Such mixed uses, however, are not discussed specifically in this paper.

2.3 Interaction Techniques

Interaction Techniques for 2D and 2.5D treemaps typically cover navigational tasks (pan, zoom, rotate), selection and

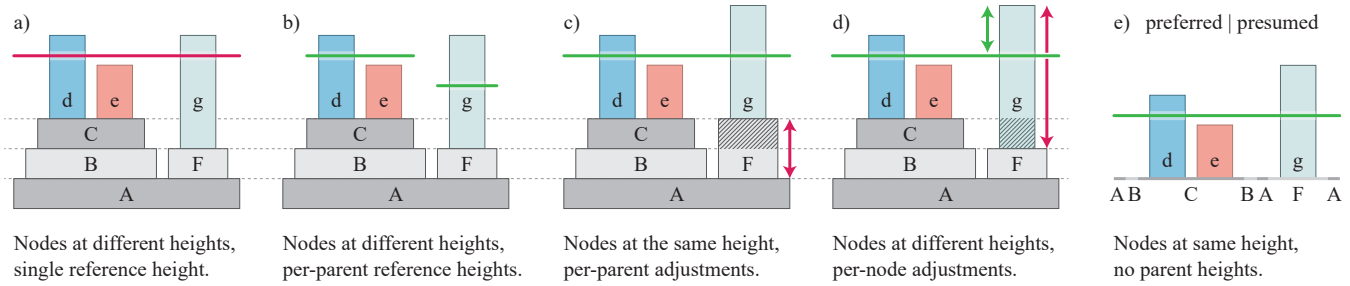


Figure 3: Variations of height reference for simultaneous use of cuboids for leaf and inner nodes (a to d). The red and green lines suggest deceptive and genuine height readout respectively. With the virtual camera's altitude presumed to be above 20° we favor the use of overlapping or nested rectangles for inner nodes (e). This reduces occlusion and visual complexity while preserving nesting clues, maintaining labeling of parents, and enabling height comparison with and without height reference.

filtering tasks (e.g., implicit or explicit queries specified via GUI elements) as well as detail providing interactions (e.g., tool tips on hovering, linking cuboids to external resources). For navigation we adhere to a basic world-in-hand metaphor as discussed for softwaremaps [19]. A comprehensive overview of interaction techniques was recently given by Jankowski and Hachet [12]. When visualizing large data sets using treemaps, handling large amounts of graphical elements renders *focus+context* and *overview+detail* techniques vital for exploration [9]. Our height-reference technique is targeting big data visualization and visual analytics tasks and is intended as an auxiliary interaction technique (similar to rotation, which, in contrast to pan and zoom, is mostly used for mitigating occlusion). With respect to the *explicit height-reference*, similar techniques were presented for "cognitive aid to augment a user's ability to compare the [height] values" in 3D histograms, such as water-level metaphor or "movable baseline" [34]. The various interactive prototypes that we implemented use interactive GPGPU-based rendering techniques as can be found in recent similar large data visualization techniques [28, 29].

2.5D treemaps are limited by a number of drawbacks [26, 32]; occlusion, underutilized screen-space, and perspective-foreshortening problems are debated continuously in the field of thematic 3D visualization [5, 24]. From a perceptual view, comparison of length in 3D projected geometry was found to be more difficult as it is in 2D [23]. Especially occlusion can lead to (1) misleading height readouts in some cases, i.e., when the junction of a leaf node's graphical element to its parent is occluded by another leaf node's element (Figure 2). Miyazaki and Itoh have presented an algorithm that reduces the overlap of piles (in a pile view, comparable to cuboids in 2.5D treemaps) by means of their bounding boxes projected onto the screen [22]. This, however, is (1) not applicable to most treemap layouting algorithms [8], (2) does not take temporal coherence on navigation into account, and (3) was only shown for a small number of piles.

In addition to occlusion, height mapping is often applied with a ludicrous data resolution, (2) exceeding the actual available pixel resolution as well as (3) overlooking the tendency of humans to overestimate the height of distant objects in comparison to objects in the foreground. The use of a height reference, as presented in this paper, focuses on mitigating these drawbacks while increasing the effectiveness of explorative use of 2.5D treemaps.

3 VISUAL DISPLAY

In this section we introduce different rendering approaches for depicting the height reference within a 2.5D treemap. To this end, we (1) state prerequisites for designing and configuring 2.5D treemaps, (2) enlist common requirements that should be accounted for, and, finally, (3) briefly describe the different rendering approaches.

3.1 Prerequisites

Due to the diversity of treemaps and their applications we made the following assumptions that should be met in order to benefit most from one of the presented rendering approaches.

- We assume the height of cuboids to change only when the data or attribute mapped changes. Specifically, we advise against height-based filtering and selection by means of modifying the height mapping (e.g., clamping and normalizing the mapped data on the fly) in order to provide consistent cuboid heights for respective interactions.
- The height mapping correlates larger heights to higher relevance for any specific task. For example, when small or negative values are relevant for a specific task, an inverted height mapping is expected (highest node).
- Data outliers are handled in some form, e.g., they are removed, a non linear mapping is used, additional visual variables are introduced, or multi-range height mapping, e.g., stacked percentiles (preferred), is used.
- The overall height scale is rather small in order reduce occlusion and promote preattentive color processing. Color, weight, structure, and labeling primarily promote any given task. Height and other visual variables take subordinate roles for details on demand and other subsequent, explorative queries.

A simultaneous use of cuboids for leaf and inner nodes (Figure 3) should be applied with care. In addition to increasing the occlusion and the overall visual complexity, it (1) suggests a meaningful height mapping for inner nodes—though commonly constant per node (as depicted in Figure 1)—and (2) aggravates height comparison even if a height reference is used. If not stated otherwise, the remainder of this paper assumes nested and flat (no height) parent nodes.

3.2 Requirements

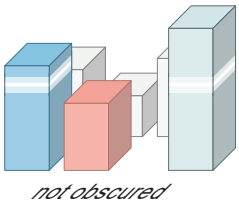
To provide feasible means for the visual display of height references in real-time we constrained our rendering approaches to several requirements. First of, the rendering should not rely on transparency or translucency. Even though it was shown that the use of transparency positively effects the perception of a visualization [13] we strive to decrease the technical needs for implementation [6] and keep the additional rendering cost to a minimum. Transparency is especially hard to solve for even the smallest treemaps (with a few hundred overlapping cuboids) and OIT (order independent transparency) rendering techniques either introduce visual artifacts such as noise or require lots of GPU memory or both. Besides that, transparency might be preferred to be used as visual variable directly already.

Parent node representations that take advantage of labeling and/or coloring should neither be fully nor partially obscured since they provide essential information regarding structural comprehension, and increase predictability of item locations within a treemap. Even though, we advise against parent heights, a rendering approach should allow for a height depiction on a per-node basis. At best, a per-fragment based rendering would not only support cuboids, but arbitrary shapes, such as sphere maps and GPU generated or tessellated geometry.

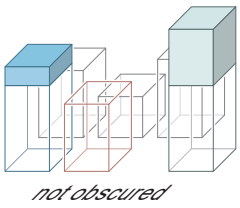
Finally, increasing the height threshold should reduce the overall visual complexity to facilitate spotting regions and nodes of interest.

3.3 Rendering

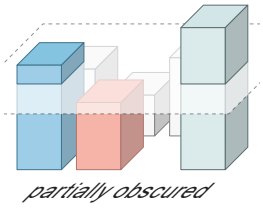
This section introduces various approaches for rendering a height reference, via intersections, an explicit surface, a closed surface, an implicit surface, stilts, and transparency bases:



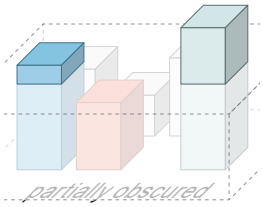
Intersections. The intersection of a surface at a height threshold is displayed using a stripe on each affected cuboid. Coloring might be orthogonal to the color scheme used for visual variable color, e.g., desaturate, lighten or darken. This approach does not obscure parent nodes and also allows for identification of nodes that fall below the threshold (all cuboids without an intersection). It is not well suited for filtering and selection though, since it hinders grouping nodes with intersections visually.



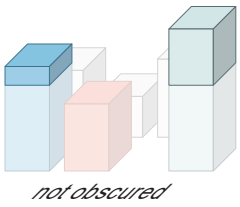
Stilts. The lower cuboid parts are cropped and replaced by one or four haloed outlines/edges (stilts) to maintain a depth cue and relation to the cuboids location. Even though parent nodes are not obscured, lots of overlapping lines are introduced and a cuboids location is harder to spot. These side effects can be reduced by rendering the lower parts transparent, but this would in turn drastically increase rendering overhead.



Explicit Surface. A transparent plane or surface is used for depicting the height reference. This, e.g., desaturates (depending on the blend mode used) the cuboid parts below the height threshold but also partially obscures parent nodes. However, depth cues imparted by screen-space based post processing—such as local ambient occlusion or unsharp-masking the depth buffer—might decreased all below the plane (and require additional rendering overhead for mitigation). Also, this approach cannot be easily applied node-locally nor does it account for parent heights.



Closed Surface. Similar to the explicit surface approach, but with additional lateral surfaces down to the bottom in order fake an infinite surface appearance. This avoids exposure of cuboid parts below the threshold at any viewing angle, but requires the surface to seamlessly integrate with the background.



Implicit Surface. All cuboids are desaturated below the height threshold on a per-fragment basis. If applied to all cuboids, the impression of a transparent, infinite surface at the height threshold is created. In contrast to the explicit or closed surface, parent nodes are not obscured, screen-space processing is not affected, and parent heights can be easily accounted for as well. Similar to the intersection approach, this can also be applied for an arbitrary sub set of nodes. In any case, the overall visual display of the treemap's structure is fully preserved.

Please note, that we favor desaturation of colors in most rendering approaches in order to reduce the saliency of node representations when falling below the height threshold. This is, however, interchangeable with other reasonable color modifications. Also note, that even though we promote the implicit surface (fragment-based) as the most favorable approach, explicit and closed surfaces (geometry-based) might be easier to integrate in certain visualization or rendering frameworks.

And, if more sophisticated rendering capabilities are available, these approaches can produce reasonable results (Figure 4).

4 INTERACTION

The height reference can be denoted as number within the domain of the attribute mapped to height. In this section, an interaction concept for manipulating the height reference using simple point/touch or drag metaphors is presented. A typical user interaction sequence for 2.5D treemaps can be characterized as follows:

(1) Gain an overview of the depicted data, understand structure, find known items or identify landmarks/nodes-of-interest.
(2) Understand the color mapping, since it is the predominant mapping and should exploit preattentive perception, i.e., locate regions-of-interest and nodes-of-interest.
(3) Only then, start exploring, e.g., w.r.t. the previously mentioned node characteristics (highest nodes, similar nodes, thresholded nodes).

During exploration the user typically uses navigation by means of pan (primary) and zoom (secondary), rotation (auxillary, for resolving occlusion) and pointing or picking in order to retain additional information display of individual nodes. For height reference manipulation we suggest to introduce an additional, auxiliary picking (and subsequent drag/scroll/swipe) that behaves slightly differently based on which face of a cuboid (top or lateral) was picked initially.

4.1 Top Cuboid Face

If the top face of a cuboid is selected (e.g., clicked/touched) for height reference, then the height reference is set to the exact height of that cuboid. This allows straight forward selection of nodes of equal or bigger height data. The height reference remains unchanged until a new top face or lateral face of any cuboid is selected (Figure 5).



Figure 4: 2.5D treemap with closed surface for height reference display using f.l.t.r. opaque, translucent, and transparent surface shading, physical based lighting, and soft shadows.

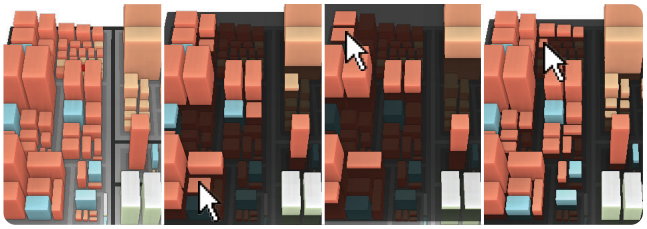


Figure 5: A small 2.5D treemap without a height reference (left). Three unrelated mouse clicks on top faces of different nodes resulting in different height references (second to fourth sub figure).

4.2 Lateral Cuboid Face

If a lateral surface of a node's representation is selected, the picked position and the normal at this position (in case of a cuboid the faces normal) are used to derive a virtual plane. The height reference is set to the height of the selected position. For all subsequent moves (while, e.g., mouse/touch remains down) the y-coordinate of the position on the plane pointed at is used as height reference. This allows for precise manipulation of height reference changes with while being locked to the interactions initial node. Without this tweak, the reference height would change abruptly whenever the node pointed at changes and disrupt the exploration process (Figure 6).

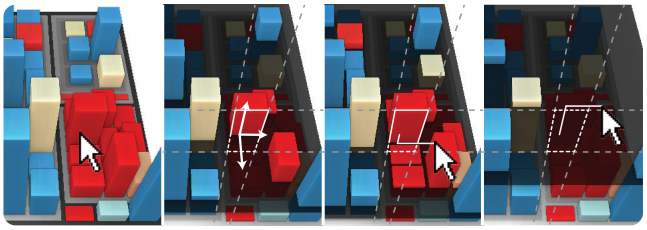


Figure 6: The red node is clicked at one of its lateral faces (left). A virtual plane of all height reference values is derived (stippled lines, starting at the second sub figure). The mouse is then moved to two positions that are intersected with the plane. The height of each respective intersection point is then used as height reference (third and fourth sub figure).

5 PRELIMINARY USER STUDY

We conducted a quantitative, preliminary user study in order to see if the use of a basic height reference has any benefits over not using it. For it, we setup the following hypotheses:

H_1 Users can identify differences in node heights faster when using the height reference (explicit surface).
H_2 Users can identify nodes of specific heights more accurate when using the height reference (explicit surface).

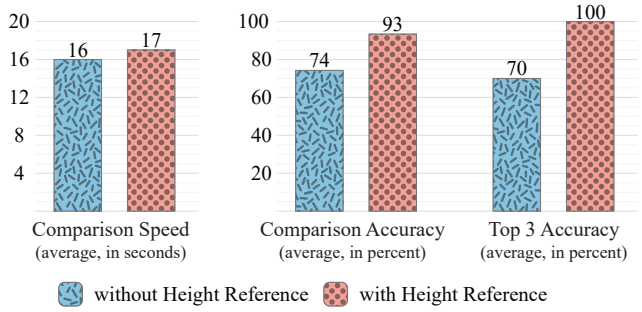
We implemented the presented technique in a prototypical research framework (C++ and OpenGL) and applied it to 2.5D bundle view, 2.5D icicle plots, as well as 2.5D treemaps.

We also helped integrate our technique into the WebGL-based 2.5D treemap visualization of a proprietary software analytics platform targeting mobile and desktop clients.

For the user study, though, we used a static web interface with pseudo interactive 2.5D treemaps featuring two specific types of interactions. First, the 2.5D treemap could not be navigated nor manipulated except for changing the height reference by dragging to discrete, precomputed positions. Second, three elements could be selected in order to select the three highest nodes.

Each participants was introduced test runs for each type of question in order to understand the interaction that are available. This was intended to reduce learning effects caused by unfamiliar interaction concepts. The study was comprised of two different tasks each with and without the use of our height reference. The first task was to pick the nodes whose height was most similar to the height of a preselected node (Figure 7). The second task asked for identification of the three highest nodes within the whole treemap. For each task, the time was measured.

The study was conducted with 20 participants with the following results:



Speed. We could not confirm our first hypothesis H_1. The average time to response for each task was slightly faster without the use of the height reference.

Accuracy. We can, however, confirm our second hypothesis H_2. The use of the height reference increase the accuarcy by 25% (similar nodes) to 42% (top 3 nodes) over the task completed without height reference.

These preliminary results indicate that height-based filtering might increase the accuracy of basic height related tasks while introducing only a marginal interaction overhead.

6 CONCLUSION AND FUTURE WORK

This paper show how 2.5D treemaps can be extended by an interactive, height-based filtering technique that facilitates accurate identification of highest nodes as well as similar nodes w.r.t. a height reference. It allows filtering and selection of nodes based on their height and depicts filtered and unselected notes in a clean way without introducing additional visual clutter. For the rendering various techniques are introduced and compared by means of technical needs, partial obscurance of structural information, visual complexity, and flexibility. The fragment-based visual display of height reference using an implicit surface is preferred since it can be implemented on a node-local basis and does not assume a specific shape of the node, as long as it supports the visual variable height. It can be applied not only for 2.5D treemaps, but for spheremaps, 2.5D icicle plots, 2.5D circular bundle views and similar visualization techniques.

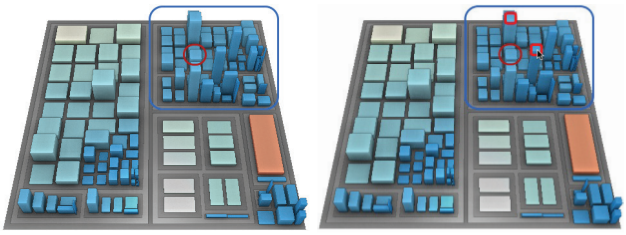


Figure 7: Initial treemap with preselected node circled red (left). Two user picks (nearest nodes) outlined red (right).

A full quantitative user study that investigates the impact of 2.5D treemap navigation metaphors mixed with the use of the presented interactive, height reference manipulation is missing and the visual quality of the various rendering approaches could be further quantified. Finally, we would like to further investigate the benefit of our technique when depicting massive amounts of data using dynamically aggregated treemaps since we expect an increase in interaction speed as well when depicting hundreds of thousands of nodes.

ACKNOWLEDGMENTS

The authors would like to thank Jonathan Herdt, Christoph Keßler, and Jan-Tobias Matysik for their preceding prototypes and valuable contributions.

REFERENCES

[1] Thomas Bladh, David A. Carr, and Jeremiah Scholl. 2004. Extending Tree-Maps to Three Dimensions: A Comparative Study. In *Proc. APCHI*. Springer, 50–59. https://doi.org/10.1007/978-3-540-27795-8_6

[2] Johannes Bohnet and Jürgen Döllner. 2011. Monitoring Code Quality and Development Activity by Software Maps. In *Proc. ACM MTD*. 9–16. https://doi.org/10.1145/1985362.1985365

[3] Abon Chaudhuri and Han-Wei Shen. 2009. A self-adaptive treemap-based technique for visualizing hierarchical data in 3D. In *Proc. IEEE PacificVis*. 105–112. https://doi.org/10.1109/PACIFICVIS.2009.4906844

[4] Steve Dübel, Martin Röhlig, Heidrun Schumann, and Matthias Trapp. 2014. 2D and 3D Presentation of Spatial Data: A Systematic Review. In *Proc. IEEE VIS Int. Workshop on 3DVis*. http://www.informatik.uni-rostock.de/~schumann/papers/2014+/survey2d3d.pdf

[5] Niklas Elmqvist and Philippas Tsigas. 2008. A Taxonomy of 3D Occlusion Management for Visualization. *IEEE TVCG* 14, 5 (2008), 1095–1109. https://doi.org/10.1109/TVCG.2008.59

[6] Eric Enderton, Erik Sintorn, Peter Shirley, and David Luebke. 2011. Stochastic Transparency. *IEEE TVCG* 17, 8 (Aug. 2011), 1036–1047. https://doi.org/10.1109/TVCG.2010.123

[7] Jean-Daniel Fekete and Catherine Plaisant. 2002. Interactive information visualization of a million items. In *Proc. IEEE INFOVIS*. 117–124. https://doi.org/10.1109/INFVIS.2002.1173156

[8] Sebastian Hahn and Jürgen Döllner. 2017. Hybrid-Treemap Layouting. In *Proc. EG EuroVis*.

[9] Helwig Hauser. 2006. Generalizing focus+context visualization. In *Scientific visualization: The visual extraction of knowledge from data*. Springer, 305–327. https://doi.org/10.1007/3-540-30790-7_18

[10] D. Holten. 2006. Hierarchical Edge Bundles: Visualization of Adjacency Relations in Hierarchical Data. *IEEE TVCG* 12, 5 (Sept 2006), 741–748. https://doi.org/10.1109/TVCG.2006.147

[11] Danny Holten, Roel Vliegen, and Jarke J. van Wijk. 2005. Visual Realism for the Visualization of Software Metrics. In *Proc. IEEE VISSOFT*. 1–6. https://doi.org/10.1109/VISSOF.2005.1684299

[12] J. Jankowski and M. Hachet. 2015. Advances in Interaction with 3D Environments. *Comput. Graph. Forum* 34, 1 (Feb. 2015), 152–190. https://doi.org/10.1111/cgf.12466

[13] J. Jankowski and M. Hachet. 2015. Advances in Interaction with 3D Environments. *Comput. Graph. Forum* 34, 1 (Feb. 2015), 152–190. https://doi.org/10.1111/cgf.12466

[14] Brian Johnson and Ben Shneiderman. 1991. Treemaps: A Space-filling Approach to the Visualization of Hierarchical Information Structures. In *Proc. IEEE VIS*. 284–291.

[15] Nicholas Kong, Jeffrey Heer, and Maneesh Agrawala. 2010. Perceptual guidelines for creating rectangular treemaps. *IEEE TVCG* (2010), 990–998.

[16] Joseph B. Kruskal and James M. Landwehr. 1983. Icicle Plot: Better Displays for Hierarchical Clustering. *The American Statistician* 37, 2 (1983), 162–168. https://doi.org/10.2307/2685881

[17] Daniel Limberger, Willy Scheibel, Sebastian Hahn, and Jürgen Döllner. 2017. Reducing Visual Complexity in Software Maps using Importance-based Aggregation of Nodes. In *Proc. IVAPP*. 176–185. https://doi.org/10.5220/0006267501760185

[18] Daniel Limberger, Willy Scheibel, Matthias Trapp, and Jürgen Döllner. 2017. Mixed-Projection Treemaps: A Novel Approach Mixing 2D and 2.5D Treemaps. In *Proc. IEEE iV*.

[19] Daniel Limberger, Benjamin Wasty, Jonas Trümper, and Jürgen Döllner. 2013. Interactive Software Maps for Web-based Source Code Analysis. In *Proc. ACM WEB3D*. 91–98. https://doi.org/10.1145/2466533.2466550

[20] Hao Lü and James Fogarty. 2008. Cascaded treemaps: examining the visibility and stability of structure in treemaps. In *Proc. GI*. Canadian Information Processing Society, 259–266.

[21] Alan M. MacEachren. 1995. *How Maps Work: representation, visualization, and design*. Guilford Press.

[22] R. Miyazaki and T. Itoh. 2009. An Occlusion-Reduced 3D Hierarchical Data Visualization Technique. In *Proc. IEEE IV*. 38–43. https://doi.org/10.1109/IV.2009.32

[23] J Farley Norman, James T Todd, Victor J Perotti, and James S Tittle. 1996. The visual perception of three-dimensional length. *Journal of Experimental Psychology: Human Perception and Performance* 22, 1 (1996), 173.

[24] Martin Röhlig and Heidrun Schumann. 2016. Visibility Widgets: Managing Occlusion of Quantitative Data in 3D Terrain Visualization. In *Proc. ACM VINCI*. 51–58.

[25] Robert E. Roth. 2016. *Visual Variables*. John Wiley & Sons, Ltd. https://doi.org/10.1002/9781118786352.wbieg0761

[26] Hans-Jorg Schulz, Steffen Hadlak, and Heidrun Schumann. 2011. The Design Space of Implicit Hierarchy Visualization: A Survey. *IEEE TVCG* 17, 4 (2011), 393–411. https://doi.org/10.1109/TVCG.2010.79

[27] Ben Shneiderman. 1992. Tree Visualization with Treemaps: A 2D Space-filling Approach. *ACM ToG* (1992), 92–99. https://doi.org/10.1145/102377.115768

[28] Avneesh Sud, Danyel Fisher, and Huai-Ping Lee. 2010. Fast dynamic voronoi treemaps. In *Proc. IEEE ISVD*. 85–94. https://doi.org/10.1109/ISVD.2010.16

[29] Matthias Trapp, Sebastian Schmechel, and Jürgen Döllner. 2013. Interactive Rendering of Complex 3D-Treemaps with a Comparative Performance Evaluation. In *Proc. GRAPP*. 165–175. https://doi.org/10.5220/0004290101650175

[30] Ying Tu and Han-Wei Shen. 2007. Visualizing Changes of Hierarchical Data using Treemaps. *IEEE TVCG* 13, 6 (2007), 1286–1293. https://doi.org/10.1109/TVCG.2007.70529

[31] Frank van Ham and Jarke J. van Wijk. 2002. Beamtrees: Compact Visualization of Large Hierarchies. In *InfoVis'02: Proceedings of the IEEE Symposium on Information Visualization*, Pak Chung Wong and Keith Andrews (Eds.). IEEE Computer Society, 93–100. https://doi.org/10.1109/INFVIS.2002.1173153

[32] Frank van Ham and Jarke J. van Wijk. 2003. Beamtrees: Compact visualization of large hierarchies. *SAGE Information Visualization* 2, 1 (2003), 31–39. https://doi.org/10.1057/palgrave.ivs.9500036

[33] Jarke J. van Wijk and Huub van de Wetering. 1999. Cushion Treemaps: Visualization of Hierarchical Information. In *Proc. IEEE INFOVIS*. 73–78. https://doi.org/10.1109/INFVIS.1999.801860

[34] Ji Soo Yi, Youn ah Kang, and John Stasko. 2007. Toward a deeper understanding of the role of interaction in information visualization. *IEEE TVCG* 13, 6 (2007), 1224–1231.